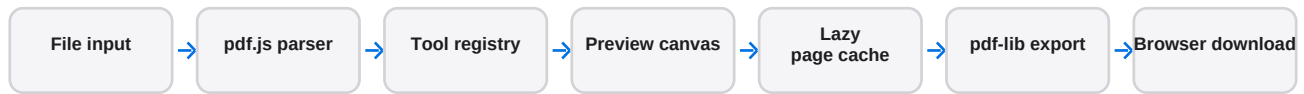


PDF Atelier

A lightweight browser-based PDF utility focused on fast everyday workflows, client-side privacy, and memory-aware rendering.



Project Snapshot

PDF Atelier is a single-page web application that packages common PDF operations into a focused tool surface: preview, organize, crop/rotate, merge, threshold, and grayscale. It is designed as an antidote to bloated desktop PDF suites: small surface area, no server upload, and careful handling of large documents on phones, tablets, and low-memory browsers.

Area	Current implementation
Stack	HTML, CSS, and vanilla JavaScript in one static page.
Libraries	pdf.js for rendering, pdf-lib for vector-preserving PDF edits, and jsPDF for raster exports.
Privacy	Files stay in the browser. No upload path or server dependency.
Large PDF handling	Lazy opening begins at 35 MB or 80 pages, with capped previews and small caches.

35 MB Large PDF trigger	80+ Safe-mode page trigger	2.2M Low-memory preview cap	3-4 Recent full-page cache
-----------------------------------	--------------------------------------	---------------------------------------	--------------------------------------

Feature Architecture

The interface is driven by a tool registry and one shared document state. Each tool is a thin mode over the same loaded PDF, which keeps the UI coherent while allowing different export behaviors.

Tool	Implementation	Why it matters
Preview	pdf.js renders the visible page to canvas; export controls are hidden.	Fast reader mode with predictable zoom and page navigation.
Organize	pdf-lib copies original pages and preserves vector content. Split actions are available from visible page controls.	Reorder, delete, and split without rasterizing the PDF.
Crop/Rotate	Crop and 90-degree rotation use PDF page boxes and rotation. The live editor preview now renders in color; fine rotation rasterizes only edited pages.	Users preview edits faithfully while preserving quality when possible.
Merge	Multiple source PDFs are loaded and pages are copied into one pdf-lib document. Merge preview uses the original-color renderer.	Merges documents without DPI options, raster conversion, or accidental black-and-white preview.
Threshold / Grayscale	Raster tools use luminance arrays and canvas pixel transforms.	Document cleanup remains explicit about preview versus export quality.

Key design decision

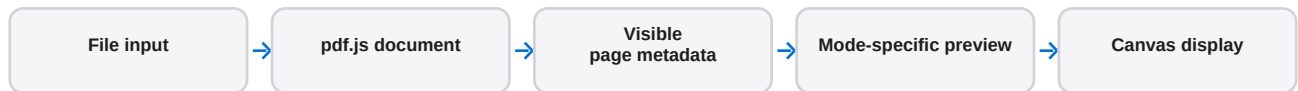
The app separates preview behavior from export behavior. Preview renders only enough pixels to display the page smoothly; export uses the chosen quality path and processes pages one at a time.

Core implementation references: `TOOLS`, `switchTool()`, `loadPdfBytes()`, `renderOriginalPreview()`, `renderEditedColorPreviewToCanvas()`, and tool-specific export functions in `index.html`.

Rendering Pipeline

PDFs are not images. To display a page inside a custom canvas UI, the visible page must be rasterized by pdf.js. The optimization is not to avoid rendering entirely, but to render the minimum useful page at the minimum useful resolution.

Preview pipeline



Mode	Preview behavior
Preview	Original color page render.
Merge	Original color page render, matching Preview rather than raster tools.
Crop/Rotate	Color page render into a temporary canvas, then crop/rotate into the editor canvas.
Threshold / Grayscale	Capped render to luminance, then pixel transform for live feedback.
Organize	Lazy thumbnails in a scrollable page grid.

Why opening still renders one page

The app renders page 1 so the user can see the document immediately. It does not need to render page 2 through page 863 during opening. That distinction is central to making very large PDFs usable in browser memory limits.

Cancellation tokens prevent stale async renders from painting over a newer selected page after fast navigation or mode switches.

Memory Strategy

The first version rendered every page up front. That is fast on a desktop for small PDFs, but it can fail on iPad or phone because a single high-DPI page can consume tens or hundreds of megabytes when expanded into canvas pixels and luminance arrays.

Problem	Current strategy
Large files	Safe mode opens lazily at 35 MB or 80 pages.
Many pages	The Crop/Rotate page strip is now a non-shrinking vertical flex list, so 800+ pages do not collapse into tiny rows.
Thumbnail quality	Organizer and editor thumbnails load low-resolution first, then upgrade through the same sequential queue.
Cache growth	Large PDFs keep only the most recent rendered luminance pages in memory while metadata and thumbnails may remain.
Preview memory	Raster and edit previews use capped screen-resolution renders; export quality is separate.

Progressive thumbnails

The page grids create a cheap thumbnail first so scrolling starts quickly. Once visible, each page is queued for a higher-quality thumbnail render. The queue is sequential to avoid a burst of 800 canvas renders.

Safe-mode cache behavior

Large PDFs keep three recent full-page luminance renders by default and four on devices reporting at least 6 GB through `navigator.deviceMemory`. Older luminance arrays are cleared while the UI keeps enough metadata to remain navigable.

Export Model

A core portfolio-worthy part of the project is that different tools preserve quality differently. Not every PDF action should rasterize a document.

Operation	Rasterizes?	Quality policy
Merge	No	Copies source PDF pages directly with pdf-lib.
Organize / Split	No	Preserves original page objects through pdf-lib.
Crop / 90-degree rotate	No	Uses crop boxes and rotation metadata.
Fine rotation	Only edited pages	Defaults to 600 dpi; Ultra uses 900 dpi.
Threshold / Grayscale	Yes	Preview is capped; export uses selected DPI page by page.

Vector-preserving exports

Merge, organize, split, crop-only, and 90-degree rotation use pdf-lib to copy or adjust original PDF page objects. This keeps text and vector graphics sharp and avoids file bloat.

Raster exports

Threshold and grayscale necessarily rasterize because they change pixels. Fine rotation also rasterizes because arbitrary degree rotation cannot be represented as a simple page metadata turn without changing visible content.

```
if no edit: copy original page
if crop or 90-degree rotation: apply vector page edit
if fine rotation: render edited page image at 600 or 900 dpi
if threshold/grayscale: render one page, process, add to output, clear canvas
```

Interaction and UI Engineering

The app is a tool surface optimized for repeated document work, not just a set of PDF functions. Recent polish focused on removing hidden behaviors and keeping custom controls accessible.

Interaction	Implementation detail
Tool navigation	Custom tabs now use tablist/tab/tabpanel semantics with roving tabindex and keyboard navigation.
Upload area	The drop zone is keyboard activatable and labeled as a button.
Split workflow	Organize pages expose visible Split controls instead of relying only on right-click.
Status feedback	Error, loader, file status, and progress updates use live/status semantics.
Zoom	Preview and Merge use original-color canvas preview with Ctrl/Cmd-scroll zoom.
Edit strip	The Crop/Rotate sidebar keeps stable page-shaped thumbnails even for 800+ page PDFs.

Performance polish

Rapid slider input is scheduled with requestAnimationFrame so the UI does not queue dozens of expensive canvas redraws. Crop/Rotate sliders update the selected preview without rebuilding the full page strip.

Engineering Tradeoffs

Browser PDF work is a balancing act: quality, memory, accessibility, and responsiveness are in tension. The implementation makes those tradeoffs explicit instead of hiding them.

Problem	Decision	Reasoning
Phones crash on giant PDFs	Lazy open plus capped preview render	Keeps the app usable without penalizing normal desktop files.
Huge page counts collapse UI	Use scrollable non-shrinking lists for the editor strip	The browser can virtual-scroll naturally instead of compressing page cards.
User expects sharp exports	Separate preview resolution from export DPI	Browsing stays light; export remains high quality.
Merge should preserve PDFs	No DPI options and color original preview	DPI only matters when rasterizing, and merge is vector-preserving.
Hidden context-menu features	Add visible Split controls	Improves discoverability on trackpads, touch devices, and keyboard workflows.

Portfolio framing

This project demonstrates browser performance debugging, memory-aware canvas rendering, client-side PDF manipulation, quality-preserving export paths, accessibility polish, and product thinking around a focused document workflow.

Talking Points for Portfolio Review

- I built a client-side PDF tool because many PDF tasks do not need a heavyweight desktop suite or server upload.
- The hardest problem was not adding buttons; it was controlling memory when large PDFs are opened in browser environments.
- I separated preview resolution from export resolution so the UI stays responsive while export quality remains explicit.
- I learned when a PDF operation should preserve vectors and when rasterization is unavoidable.
- Recent iterations fixed real usability issues: keyboard access, visible split controls, color previews in Merge and Crop/Rotate, progressive thumbnails, and stable editor cards for 800+ page files.

Suggested demo sequence

1. Open a large PDF in Preview and show that it loads without rendering every page.
2. Switch to Organize and explain vector-preserving reorder/split plus progressive thumbnails.
3. Use Crop/Rotate and explain color preview versus vector-preserving export.
4. Switch to Threshold/Grayscale and explain capped preview versus high-DPI export.

Implementation file: `index.html`. Main libraries: `pdf.js` for rendering, `pdf-lib` for vector-preserving page operations, and `jsPDF` for raster PDF output.